

TMDB Weekly Trends ETL Pipeline

Local and AWS Walkthrough

Overview

- This project fetches weekly trending movie data from TMDB, enriches it with movie details, converts the dataset to parquet, and stores it in Amazon S3.
- The project now supports two execution modes:
- Local mode with Apache Airflow and Astro for orchestration and debugging.
- AWS mode with Lambda and EventBridge Scheduler for low-cost weekly cloud execution.

Project flow in simple words

- TMDB API provides the trending movies.
- Shared ETL code fetches and transforms the data.
- The final dataset is written as a parquet file to Amazon S3.
- Airflow is used locally to orchestrate the workflow.
- Lambda is used in AWS to execute the same ETL logic on a schedule.

Part 1: What was done locally

Step 1: Created the Astro and Airflow project

- Started from an Astro Airflow project structure.
- Kept the DAG inside the dags folder so Airflow could discover it.
- Used Docker Desktop and Astro CLI to run Airflow locally.

Step 2: Built the ETL workflow

- Created a DAG called `tmdb_weekly_trending_movies`.
- Added three task stages:
- `extract_trending_movies`
- `transform_movie_details`
- `load_weekly_snapshot`
- The pipeline fetches trending movies from TMDB, enriches each movie using the details endpoint, and stores the weekly snapshot in S3.

Step 3: Configured TMDB authentication

- At first, the code used the TMDB `api_key` query parameter.
- The TMDB credential being used was actually a Bearer token.
- The DAG was updated to send the TMDB token in the Authorization header.
- The Airflow Variable name was changed to `tmdb_access_token`.

Step 4: Configured S3 for local Airflow

- Created the Airflow connection `aws_default`.
- Added bucket and prefix variables in Airflow:
- `tmdb_access_token`
- `tmdb_s3_bucket`
- `tmdb_s3_prefix_weekly`
- `tmdb_aws_conn_id`
- Local Airflow upload uses S3Hook and the Airflow AWS connection.

Step 5: Fixed local environment issues

- Rebuilt the broken Python virtual environment.
- Cleaned the requirements file so Astro Runtime could build properly.
- Added the missing `apache-airflow-providers-amazon` package for S3Hook.
- Updated `.dockerignore` so Docker would not include temporary pytest cache folders.

Step 6: Ran the DAG locally

- Started Airflow with `astro dev start`.
- Verified the DAG in the Airflow UI.
- Triggered manual DAG runs.
- Confirmed the parquet output was written to S3.

Part 2: Why the project was adapted for AWS

Problem

- Running Airflow locally is great for development, but it is not ideal as a weekly always-on production scheduler for a portfolio project.
- Managed Airflow services are not free, and always-on infrastructure is unnecessary for one small weekly ETL.

Solution

- Kept Airflow for local orchestration and demonstration.
- Added a second cloud execution path using AWS Lambda plus EventBridge Scheduler.
- This keeps the project professional while staying cheaper and simpler.

Part 3: What was changed for AWS

Step 1: Extracted shared ETL logic

- Moved reusable ETL logic into `src/tmdb_pipeline.py`.

- This shared module now contains:
- TMDB API calls
- record enrichment logic
- parquet conversion
- S3 key generation
- boto3-based S3 upload

Step 2: Kept Airflow behavior intact

- The Airflow DAG still works locally as before.
- It now imports shared ETL functions instead of keeping all logic inside the DAG file.
- Local upload still uses Airflow S3Hook and Airflow Variables.

Step 3: Added Lambda entrypoint

- Created `lambda_handler.py`.
- Lambda reads its configuration from environment variables instead of Airflow Variables.
- Lambda calls the same shared ETL module as Airflow.

Step 4: Added Lambda packaging files

- Created `requirements-lambda.txt`.
- Created `Dockerfile.lambda`.
- Used a Lambda container image because pandas and pyarrow are easier to package that way.

Part 4: AWS deployment steps

Step 1: Created an ECR repository

- Repository name used:
- `tmdb-weekly-trends-lambda`
- Region used:
- `us-east-1` (N. Virginia)

Step 2: Built and pushed the Lambda image

- Built the image locally with Docker.
- Pushed the image to Amazon ECR.
- Rebuilt using a Lambda-compatible manifest because the first image format was not accepted by Lambda.

Step 3: Created the Lambda function

- Created a Lambda function from the pushed ECR image.
- Configured:

- architecture x86_64
- timeout 3 minutes
- memory 1024 MB
- Added environment variables:
- TMDB_ACCESS_TOKEN
- S3_BUCKET
- S3_PREFIX

Step 4: Added IAM permissions

- Added an execution role with CloudWatch logging.
- Added S3 permissions so Lambda can:
- list the bucket
- get bucket location
- put objects into the bucket

Step 5: Tested Lambda manually

- Invoked the Lambda function using a simple test event.
- Fixed packaging and image issues until the function executed successfully.
- Confirmed the Lambda run also wrote parquet output to S3.

Part 5: Weekly schedule in AWS

Step 1: Created EventBridge Scheduler

- Used EventBridge Scheduler instead of older EventBridge scheduled rules.
- Chose a recurring cron-based schedule.

Step 2: Configured weekly timing

- Final business schedule chosen:
- Every Friday at 6:00 PM US Eastern Time
- Timezone used:
- America/New_York
- Cron expression:
- 0 18 ? * FRI *

Step 3: Connected the target

- EventBridge Scheduler invokes the working Lambda function.
- AWS created or attached the permission role needed for the scheduler to call Lambda.

Part 6: Final architecture

Local mode

- Developer runs `astro dev start`.
- Airflow DAG executes the ETL.
- Shared ETL code processes TMDB data.
- S3Hook uploads parquet to S3.

AWS mode

- EventBridge Scheduler triggers Lambda every Friday at 6 PM Eastern Time.
- Lambda executes the shared ETL code.
- boto3 uploads parquet to S3.
- CloudWatch stores the logs.

Part 7: Portfolio value

- Shows Apache Airflow orchestration skills.
- Shows Docker and Astro local development workflow.
- Shows AWS Lambda, ECR, EventBridge Scheduler, IAM, and S3 integration.
- Shows practical system design with two execution modes:
 - local orchestration
 - cloud scheduled execution

Part 8: Key lessons learned

- Airflow is excellent for orchestration and local workflow visibility.
- For a small weekly ETL, Lambda plus EventBridge is cheaper and easier to keep running than managed Airflow.
- Shared ETL logic makes it possible to support both Airflow and AWS Lambda without duplicating the business logic.
- Using timezone-aware scheduling is important when defining weekly automated jobs.

Part 9: Recommended next improvements

- Add a clean README for GitHub.
- Add an architecture diagram.
- Add a short demo video showing local and AWS execution.
- Rotate any exposed AWS credentials and secrets.
- Add alerting and data quality checks for production-style polish.